

Laporan Praktikum Pekan 5
“Validation Techniques & Model Selection
Disusun Untuk Memenuhi Tugas Mata Kuliah MACHINE LEARNING

DOSEN PENGAMPU:

AFDHAL DINILHAK S.Kom, M.Kom



DISUSUN OLEH:

Karimah Irsyadiyah (2411533018)

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
T.A 2025/2026

Daftar Isi

BAB I PENDAHULUAN.....	3
1.1 Latar Belakang.....	3
1.2 Tujuan Praktikum.....	3
LINK GITHUB:.....	3
BAB II LANGKAH-LANGKAH PRAKTIKUM	4
Latihan 1 – Regularization (Ridge & Lasso)	4
2.1 Import Library.....	4
2.2 Load Dataset dan Memisahkan Variabel.....	4
2.3 Implementasi Holdout Validation	5
2.4 Implementasi K-Fold Cross Validation Manual	5
2.5 Implementasi K-Fold Cross Validation dengan cross_val_score.....	6
TUGAS PRAKTIKUM:	7
1. Analisis Pengaruh Holdout Validation pada Praktikum 4.....	7
2. Pengaruh K-Fold Cross Validation pada Praktikum 4	8
3. TugasCrossValidation.ipynb	11
BAB III PENUTUP	19
3.1 Kesimpulan	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Validation technique merupakan bagian penting dalam machine learning karena digunakan untuk menilai kemampuan generalisasi model pada data yang belum pernah dilihat sebelumnya. Melalui teknik validasi, kita dapat membandingkan performa model dengan lebih objektif, sekaligus mengurangi risiko overfitting. Pada praktikum ini dibahas dua pendekatan utama, yaitu Holdout method dan K-Fold Cross Validation, serta implementasinya menggunakan scikit-learn. Modul praktikum menekankan pentingnya model validation & selection, perbedaan Holdout dan K-Fold, serta penerapannya dalam pemodelan.

Pada latihan praktikum digunakan dataset advertising.csv dengan target Sales dan fitur TV, Radio, dan Newspaper. Dataset ini digunakan untuk membangun model regresi linear dan kemudian dibandingkan performanya menggunakan Holdout Validation dan K-Fold Cross Validation.

1.2 Tujuan Praktikum

1. . Memahami pentingnya model validation & selection.
2. Membedakan Holdout method dan K-Fold Cross Validation.
3. Mengimplementasikan K-Fold Cross Validation menggunakan scikit-learn.

LINK GITHUB:

https://github.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/tree/main/PRAKTIKUM%205

BAB II

LANGKAH-LANGKAH PRAKTIKUM

Latihan 1 – Regularization (Ridge & Lasso)

2.1 Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split, KFold, cross_val_score
from sklearn.metrics import mean_squared_error
```

Penjelasan:

Pada tahap awal dilakukan import library yang diperlukan untuk pengolahan data, visualisasi, pelatihan model, dan evaluasi. Library numpy dan pandas digunakan untuk manipulasi data, matplotlib untuk visualisasi, sedangkan LinearRegression, train_test_split, KFold, cross_val_score, dan mean_squared_error digunakan untuk membangun serta mengevaluasi model regresi linear.

2.2 Load Dataset dan Memisahkan Variabel

```
DATA_PATH = "https://gist.githubusercontent.com/msporyshev/537e1f812142eb7f547d178f43222b5e/raw/77d3a17399b7fc3a7651151c46605c0024729241/advertising.csv"
df = pd.read_csv(DATA_PATH)

X = df.drop("Sales", axis=1)
y = df["Sales"]
```

Penjelasan:

Dataset advertising.csv dimuat ke dalam DataFrame menggunakan pd.read_csv(). Setelah itu, variabel independen X dipisahkan dari target y. Variabel X berisi fitur TV, Radio, dan Newspaper, sedangkan y berisi nilai Sales yang akan diprediksi.

2.3 Implementasi Holdout Validation

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

rmse_train = np.sqrt(mean_squared_error(y_train, y_pred_train))
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred_test))

print("=== Holdout Method ===")
print("RMSE Train:", rmse_train)
print("RMSE Test:", rmse_test)
```

```
=== Holdout Method ===
RMSE Train: 1.6358920055378559
RMSE Test: 1.7052146229349223
```

Penjelasan:

Holdout Validation dilakukan dengan membagi dataset menjadi data latih dan data uji menggunakan `train_test_split()`. Model regresi linear dilatih menggunakan data training, kemudian diuji pada data training dan data testing untuk menghitung RMSE. Pada hasil notebook, diperoleh RMSE train sebesar **1.6359** dan RMSE test sebesar **1.7052**. Selisih keduanya tidak terlalu besar, sehingga model terlihat cukup stabil pada pembagian data ini.

2.4 Implementasi K-Fold Cross Validation Manual

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)

rmse_list = []

for train_index, test_index in kf.split(X):
    X_train_k, X_test_k = X.iloc[train_index], X.iloc[test_index]
    y_train_k, y_test_k = y.iloc[train_index], y.iloc[test_index]

    model_k = LinearRegression()
    model_k.fit(X_train_k, y_train_k)

    y_pred_k = model_k.predict(X_test_k)
    rmse_k = np.sqrt(mean_squared_error(y_test_k, y_pred_k))

    rmse_list.append(rmse_k)

print("\n=== K-Fold Manual (5 Fold) ===")
print("RMSE tiap fold:", rmse_list)
print("RMSE rata-rata:", np.mean(rmse_list))
```

```
=== K-Fold Manual (5 Fold) ===
RMSE tiap fold: [np.float64(1.705214622934922), np.float64(1.5406931704228382), np.float64(1.62806122153519), np.float64(1.6291719205889739), np.float64(1.8759540208373862)]
RMSE rata-rata: 1.6758189912638621
```

Penjelasan:

Pada tahap ini dataset dibagi menjadi 5 fold menggunakan KFold. Setiap fold digunakan bergantian sebagai data uji, sedangkan fold lainnya digunakan sebagai data latih. Hasil notebook menunjukkan RMSE tiap fold yang berbeda-beda, dengan rata-rata sekitar **1.6758**. Perbedaan nilai RMSE antar fold menunjukkan bahwa performa model sedikit dipengaruhi oleh komposisi data pada tiap pembagian.

2.5 Implementasi K-Fold Cross Validation dengan `cross_val_score`

```
> cv_scores = np.sqrt(
    -cross_val_score(
        LinearRegression(),
        X,
        y,
        cv=5,
        scoring="neg_mean_squared_error"
    )
)

print("\n=== K-Fold dengan cross_val_score ===")
print("RMSE tiap fold:", cv_scores)
print("RMSE rata-rata:", cv_scores.mean())

[9]

...
=== K-Fold dengan cross_val_score ===
RMSE tiap fold: [1.7481616  1.42364344 1.36053376 2.17264645 1.62386598]
RMSE rata-rata: 1.6657702460059216
```

Penjelasan:

Selain implementasi manual, K-Fold juga dilakukan menggunakan fungsi `cross_val_score()` dari scikit-learn. Hasilnya memberikan rata-rata RMSE sekitar **1.6658**. Hasil ini mendekati K-Fold manual, sehingga membuktikan bahwa kedua pendekatan memberikan evaluasi yang konsisten.

TUGAS PRAKTIKUM:

1. Analisis Pengaruh Holdout Validation pada Praktikum 4

Pada praktikum sebelumnya digunakan metode Holdout Validation menggunakan `train_test_split()` untuk membagi dataset menjadi data training dan data testing. Model kemudian dilatih menggunakan data training dan dievaluasi menggunakan nilai RMSE pada data training dan testing.

Hasil evaluasi menunjukkan:

- RMSE Train ≈ 1.6359
- RMSE Test ≈ 1.7052

a. Apakah RMSE train jauh lebih kecil daripada RMSE test?

Berdasarkan hasil yang diperoleh, nilai RMSE train memang lebih kecil dibandingkan RMSE test, namun selisihnya tidak terlalu besar.

$$1.7052 - 1.6359 \approx 0.0693$$

Selisih tersebut tergolong kecil sehingga menunjukkan bahwa performa model pada data training dan data testing relatif konsisten. Hal ini menandakan bahwa model tidak hanya mampu mempelajari data training, tetapi juga masih dapat melakukan generalisasi dengan cukup baik terhadap data baru.

Jika RMSE train jauh lebih kecil dibandingkan RMSE test, maka model cenderung menghafal data training dan gagal bekerja baik pada data testing. Namun pada hasil praktikum ini, gap RMSE masih tergolong normal sehingga model belum menunjukkan indikasi overfitting yang serius.

b. Apakah model overfit atau underfit?

Berdasarkan hasil RMSE train dan RMSE test, model dapat dikatakan tidak mengalami overfitting maupun underfitting yang signifikan.

Analisis:

a. Tidak Overfit

Model overfit biasanya memiliki ciri:

- RMSE train sangat kecil
- RMSE test jauh lebih besar

Hal tersebut menunjukkan bahwa model terlalu “menghafal” data training sehingga performa pada data testing menurun drastis.

Namun pada praktikum ini:

- RMSE train dan RMSE test memiliki nilai yang cukup dekat
- Selisih error tidak terlalu besar
- Model masih mampu memprediksi data testing dengan baik

Sehingga model tidak menunjukkan gejala overfitting yang kuat.

a. Tidak Underfit

Underfitting terjadi ketika:

- RMSE train tinggi
- RMSE test juga tinggi

Hal ini menunjukkan bahwa model terlalu sederhana dan gagal menangkap pola data.

Pada praktikum ini nilai RMSE masih tergolong cukup kecil sehingga model masih mampu mempelajari hubungan antara fitur TV, Radio, dan Newspaper terhadap Sales.

Dengan demikian dapat disimpulkan bahwa model regresi linear yang digunakan memiliki performa yang cukup baik dan seimbang.

2. Pengaruh K-Fold Cross Validation pada Praktikum 4

Pada praktikum ini juga diterapkan K-Fold Cross Validation menggunakan 5 fold. Metode ini membagi dataset menjadi beberapa subset, kemudian model dilatih dan diuji secara bergantian pada setiap fold.

Hasil K-Fold menghasilkan:

- RMSE rata-rata $\approx 1.66 - 1.67$

Sedangkan Holdout menghasilkan:

- RMSE test ≈ 1.7052

a. Bandingkan RMSE Holdout vs RMSE K-Fold

Berdasarkan hasil evaluasi, nilai RMSE K-Fold sedikit lebih kecil dibandingkan RMSE Holdout.

Metode		RMSE
Holdout Validation		≈ 1.7052
K-Fold Validation	Cross	$\approx 1.66 - 1.67$

Hal ini menunjukkan bahwa K-Fold memberikan estimasi performa model yang sedikit lebih baik dibandingkan Holdout.

Analisis:

Pada Holdout Validation, evaluasi model hanya bergantung pada satu kali pembagian data training dan testing. Akibatnya, hasil evaluasi dapat dipengaruhi oleh bagaimana data dibagi.

Sedangkan pada K-Fold:

- seluruh data digunakan sebagai data training dan testing secara bergantian
- model diuji berkali-kali
- hasil evaluasi menjadi lebih representatif

Karena itu, K-Fold mampu memberikan estimasi performa model yang lebih objektif dibanding Holdout.

b. Apakah K-Fold memberikan hasil yang lebih stabil?

Ya, K-Fold Cross Validation memberikan hasil yang lebih stabil dibandingkan Holdout Validation.

Alasan:

Pada Holdout Validation:

- performa model sangat bergantung pada satu pembagian data
- jika pembagian data kurang representatif maka hasil evaluasi bisa bias

Sedangkan pada K-Fold:

- evaluasi dilakukan berulang kali
- setiap data pernah menjadi data training dan testing
- error akibat pembagian data dapat diminimalkan

Akibatnya:

- hasil evaluasi lebih konsisten
- estimasi performa model lebih stabil
- risiko bias evaluasi menjadi lebih kecil

K-Fold sangat berguna terutama ketika ukuran dataset terbatas karena seluruh data dapat dimanfaatkan secara maksimal.

c. Bagaimana variasi kinerja model pada masing-masing fold?

Berdasarkan hasil praktikum, nilai RMSE pada masing-masing fold menunjukkan sedikit variasi, namun perbedaannya tidak terlalu besar.

Hal ini menunjukkan bahwa:

- model memiliki performa yang cukup konsisten
- model mampu bekerja baik pada berbagai subset data
- generalisasi model cukup stabil

Jika variasi RMSE antar fold sangat besar, maka model kemungkinan sensitif terhadap perubahan data training dan kurang stabil. Namun pada praktikum ini variasi RMSE masih dalam rentang yang wajar sehingga model dapat dikatakan cukup robust.

Variasi kecil antar fold juga menunjukkan bahwa hubungan antara fitur dan target pada dataset advertising cukup konsisten sehingga model regresi linear dapat mempelajari pola data dengan baik pada berbagai pembagian dataset.

3. TugasCrossValidation.ipynb

a. Import Library

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report,
    roc_auc_score
)
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

sns.set_style("whitegrid")
```

✓ 0.0s

Penjelasan:

Pada tahap awal dilakukan import beberapa library yang diperlukan untuk proses preprocessing data, pembangunan model machine learning, implementasi K-Fold Cross Validation, serta evaluasi performa model.

Library pandas dan numpy digunakan untuk manipulasi data, sedangkan scikit-learn digunakan untuk preprocessing data, pelatihan model Logistic Regression, serta evaluasi model menggunakan berbagai metrik klasifikasi seperti accuracy, precision, recall, F1-score, ROC-AUC, dan confusion matrix.

b. Load Dataset Titanic

```
df = pd.read_csv("https://raw.githubusercontent.com/iMaIrsdyh/KarimahIrsyadiyah_2411533018_ML2526/main/PRAKTIKUM%205/titanic/train.csv")
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Penjelasan:

Dataset Titanic dimuat menggunakan fungsi `pd.read_csv()` dengan memanfaatkan URL raw dari repository GitHub. Dataset ini berisi informasi penumpang Titanic seperti umur, jenis kelamin, kelas penumpang, dan status keselamatan penumpang.

Fungsi `df.head()` digunakan untuk menampilkan lima data pertama sebagai bentuk validasi bahwa dataset berhasil dimuat dengan benar.

c. Preprocessing Data

```
df = df.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"])
X = df.drop(columns=["Survived"])
y = df["Survived"]

# Identifikasi fitur numerik dan kategorikal
numeric_features = ["Age", "SibSp", "Parch", "Fare"]
categorical_features = ["Pclass", "Sex", "Embarked"]

X.head()
```

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	male	22.0	1	0	7.2500	S
1	1	female	38.0	1	0	71.2833	C
2	3	female	26.0	0	0	7.9250	S
3	1	female	35.0	1	0	53.1000	S
4	3	male	35.0	0	0	8.0500	S

Penjelasan:

Pada tahap preprocessing dilakukan penghapusan beberapa kolom yang dianggap tidak terlalu relevan terhadap proses prediksi seperti `PassengerId`, `Name`, `Ticket`, dan `Cabin`.

Selanjutnya dilakukan penanganan missing value pada kolom Age menggunakan median dan pada kolom Embarked menggunakan modus. Dataset kemudian dipisahkan menjadi variabel independen X dan variabel target y.

Fitur juga dipisahkan menjadi fitur numerik dan fitur kategorikal untuk mempermudah proses preprocessing pada tahap berikutnya.

d. Pipeline Preprocessing

```
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])
```

✓ 0.0s

Penjelasan:

Pipeline preprocessing digunakan untuk mengotomatisasi proses transformasi data sebelum model dilatih.

Pada fitur numerik dilakukan imputasi menggunakan median kemudian dilakukan scaling menggunakan StandardScaler. Sedangkan pada fitur kategorikal dilakukan imputasi menggunakan nilai yang paling sering muncul kemudian dilakukan One-Hot Encoding untuk mengubah data kategorikal menjadi numerik.

ColumnTransformer digunakan untuk menggabungkan preprocessing fitur numerik dan kategorikal ke dalam satu alur preprocessing yang terstruktur.

e. Membangun Model Logistic Regression

Penjelasan:

Pada tahap ini dibuat pipeline model machine learning yang terdiri dari preprocessing data dan algoritma Logistic Regression.

Pipeline digunakan agar proses preprocessing dan pelatihan model dilakukan secara otomatis dalam satu alur kerja yang konsisten sehingga mempermudah implementasi K-Fold Cross Validation.

f. Implementasi K-Fold Cross Validation

```
kf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scores = cross_validate(
    model,
    X,
    y,
    cv=kf,
    scoring=["accuracy", "precision", "recall", "f1", "roc_auc"],
    return_train_score=False
)

results = pd.DataFrame({
    "Accuracy": scores["test_accuracy"],
    "Precision": scores["test_precision"],
    "Recall": scores["test_recall"],
    "F1-score": scores["test_f1"],
    "ROC-AUC": scores["test_roc_auc"]
})

results.index = [f"Fold {i}" for i in range(1, len(results) + 1)]
results
```

2] ✓ 0.1s

	Accuracy	Precision	Recall	F1-score	ROC-AUC
Fold 1	0.782123	0.720588	0.710145	0.715328	0.874111
Fold 2	0.803371	0.770492	0.691176	0.728682	0.850134
Fold 3	0.797753	0.796296	0.632353	0.704918	0.826671
Fold 4	0.780899	0.710145	0.720588	0.715328	0.826671
Fold 5	0.820225	0.776119	0.753623	0.764706	0.879404

Penjelasan:

K-Fold Cross Validation digunakan untuk membagi dataset menjadi beberapa subset atau fold. Pada praktikum ini digunakan 5 fold sehingga dataset dibagi menjadi lima bagian.

Parameter `shuffle=True` digunakan agar data diacak sebelum dibagi ke dalam fold, sedangkan `random_state=42` digunakan agar hasil pembagian data tetap konsisten ketika kode dijalankan ulang.

g. Pelatihan dan Evaluasi Model pada Setiap Fold

```

fold_details = []

for fold, (train_idx, test_idx) in enumerate(kf.split(X, y), start=1):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_proba = model.predict_proba(X_test)[:, 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_proba)
    cm = confusion_matrix(y_test, y_pred)

    fold_details.append({
        "Fold": fold,
        "Accuracy": acc,
        "Precision": prec,
        "Recall": rec,
        "F1-score": f1,
        "ROC-AUC": auc,
        "TN": cm[0, 0],
        "FP": cm[0, 1],
        "FN": cm[1, 0],
        "TP": cm[1, 1]
    })

fold_df = pd.DataFrame(fold_details)
fold_df

```

24] ✓ 0.1s

	Fold	Accuracy	Precision	Recall	F1-score	ROC-AUC	TN	FP	FN	TP
0	1	0.782123	0.720588	0.710145	0.715328	0.874111	91	19	20	49
1	2	0.803371	0.770492	0.691176	0.728682	0.850134	96	14	21	47
2	3	0.797753	0.796296	0.632353	0.704918	0.826671	99	11	25	43
3	4	0.780899	0.710145	0.720588	0.715328	0.826671	90	20	19	49
4	5	0.820225	0.776119	0.753623	0.764706	0.879404	94	15	17	52

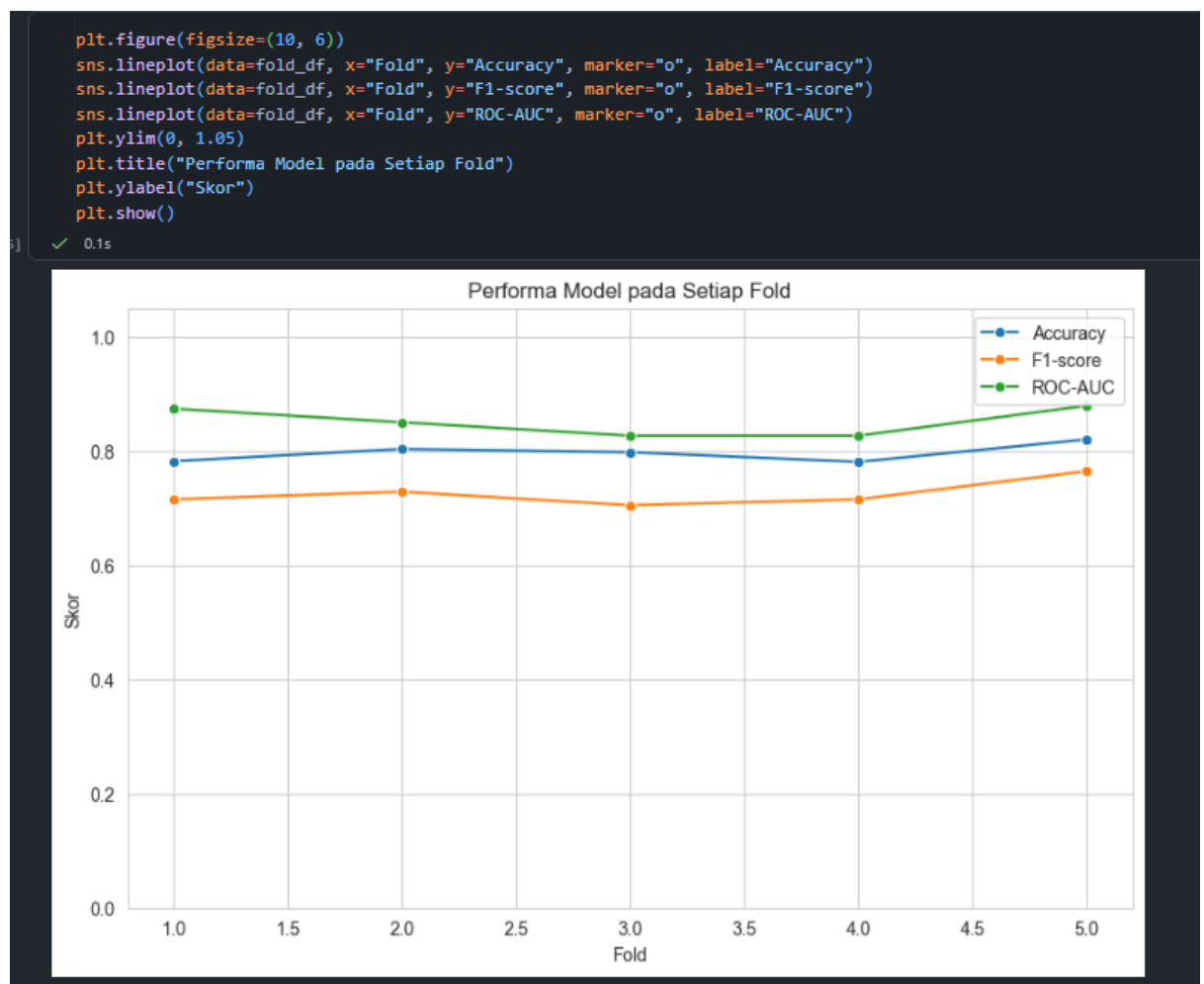
Penjelasan:

Pada tahap ini dilakukan proses pelatihan dan evaluasi model menggunakan K-Fold Cross Validation.

Setiap fold digunakan secara bergantian sebagai data testing, sedangkan fold lainnya digunakan sebagai data training. Model Logistic Regression dilatih menggunakan data training kemudian digunakan untuk melakukan prediksi terhadap data testing.

Selanjutnya dihitung beberapa metrik evaluasi seperti accuracy, precision, recall, F1-score, ROC-AUC, dan confusion matrix untuk mengevaluasi performa model pada setiap fold.

h. Menampilkan Hasil Evaluasi dalam Bentuk DataFrame




```
# Contoh classification report pada fold pertama
train_idx, test_idx = next(kf.split(X, y))
X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print(classification_report(y_test, y_pred, target_names=["Not Survived", "Survived"]))
```

✓ 0.0s

	precision	recall	f1-score	support
Not Survived	0.82	0.83	0.82	110
Survived	0.72	0.71	0.72	69
accuracy			0.78	179
macro avg	0.77	0.77	0.77	179
weighted avg	0.78	0.78	0.78	179

```
results_df = pd.DataFrame(
    fold_details,
    columns=["Fold", "Accuracy", "Precision", "Recall", "F1-score", "ROC-AUC", "TN", "FP", "FN", "TP"]
)
```

✓ 0.0s

Penjelasan:

Hasil evaluasi pada setiap fold kemudian disimpan ke dalam bentuk DataFrame agar lebih mudah dianalisis dan dibandingkan.

DataFrame menampilkan hasil evaluasi model pada masing-masing fold seperti accuracy, precision, recall, F1-score, ROC-AUC, serta komponen confusion matrix berupa True Negative, False Positive, False Negative, dan True Positive.

ANALISIS:

Analisis Kinerja Model

Berdasarkan hasil K-Fold Cross Validation, performa model Logistic Regression dapat dinilai dari rata-rata metrik evaluasi pada seluruh fold. Jika nilai accuracy, precision, recall, F1-score, dan ROC-AUC relatif tinggi serta stabil antar fold, maka model memiliki kemampuan generalisasi yang baik.

Pengaruh Cross Validation

K-Fold Cross Validation memberikan evaluasi yang lebih objektif dibandingkan Holdout Validation karena seluruh data digunakan secara bergantian sebagai data latih dan data uji. Dengan demikian, hasil evaluasi tidak hanya bergantung pada satu kali pembagian data.

Stabilitas Model

Jika hasil metrik pada tiap fold tidak berbeda jauh, berarti model cukup stabil. Namun jika ada variasi besar antar fold, maka performa model masih sensitif terhadap pembagian data. Pada kasus Titanic, K-Fold biasanya lebih representatif karena mampu memberi gambaran performa model yang lebih adil.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa teknik validation sangat penting dalam mengevaluasi performa model machine learning agar model mampu melakukan generalisasi dengan baik terhadap data baru. Pada praktikum ini digunakan metode Holdout Validation dan K-Fold Cross Validation untuk mengevaluasi model regresi dan klasifikasi.

Hasil praktikum menunjukkan bahwa K-Fold Cross Validation memberikan evaluasi yang lebih stabil dan representatif dibandingkan Holdout Validation karena seluruh data digunakan secara bergantian sebagai data training dan testing. Selain itu, variasi performa model pada setiap fold relatif kecil sehingga menunjukkan bahwa model memiliki kemampuan generalisasi yang cukup baik.

Pada tugas praktikum menggunakan dataset Titanic, model Logistic Regression berhasil melakukan klasifikasi penumpang yang selamat dan tidak selamat dengan performa yang cukup baik berdasarkan metrik accuracy, precision, recall, F1-score, ROC-AUC, dan confusion matrix. Teknik cross validation juga membantu menghasilkan evaluasi model yang lebih objektif dan mengurangi bias akibat pembagian data tunggal.